AgroSmart – Análisis de Algoritmos y Estructuras de Datos

Módulo: Riego y plan de agua (riego_agrosmart.py, planificador.py y pantalla_riego.py)

1. Cómo está organizado el módulo

La pantalla **Riego** (`pantalla_riego.py`) junta dos partes que comparten los mismos cultivos del módulo Cultivos:

1. **Riego con sensor (simulado)** – `riego_agrosmart.py`: para cada cultivo calcula umbrales de humedad, lee un sensor simulado, consulta un clima simulado y decide si abre la válvula.
2. **Plan de agua** – `planificador.py`: recomienda litros y frecuencia de riego, lleva la lista de riegos pendientes (cola) y hechos (lista enlazada), reparte el agua disponible (algoritmo voraz) y calcula distancias y recorridos entre parcelas (grafo y Dijkstra).

Estructura o algoritmo	Dónde	Para qué
Diccionario (tabla hash)	`self.zonas`	Encuentra un cultivo por su código.
Cola	`estructuras/cola.py`	Riegos pendientes.
Lista enlazada	`estructuras/lista_enlazada.py`	Historial de riegos hechos.
Grafo	`estructuras/grafo.py`	Distancias en km entre la casa y las parcelas.
Dijkstra	`algoritmos/dijkstra.py`	Camino más corto.
Voraz + Merge Sort	`algoritmos/voraz_agua.py`	Repartir agua limitada.

Nota: el sensor, la válvula y el clima de esta pantalla son una **simulación**, porque el sistema no está conectado a aparatos reales. Los litros del sensor son valores de ejemplo; los litros del plan recomendado se calculan según el tamaño real de la parcela.

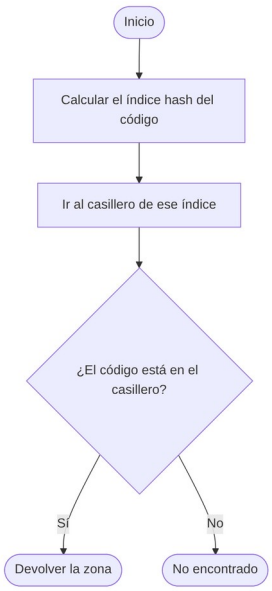
2. Búsqueda de cultivo por código (tabla hash)

Explicación. Cada cultivo se guarda en un diccionario de Python (`self.zonas`) con su código (por ejemplo “CULT001”) como llave. Un diccionario funciona como una tabla hash: calcula un número a partir de la llave y va directo al casillero, sin recorrer los demás cultivos. Se usa en `buscar_por_codigo`, `registrar_cultivo` y `eliminar_cultivo`.

Pseudocódigo

```
FUNCION buscar_por_codigo(zonas, codigo)
    indice = FUNCION_HASH(codigo)
    casillero = zonas.casilleros[indice]
    SI codigo EXISTE en casillero: DEVOLVER casillero[codigo]
    SINO: DEVOLVER NADA
```

Diagrama de flujo



Complejidad. Tiempo O(1) en promedio, O(n) en el peor caso. Espacio O(n).

Justificación. El código es el identificador único de cada cultivo y estas operaciones se repiten cada vez que se revisa el riego.

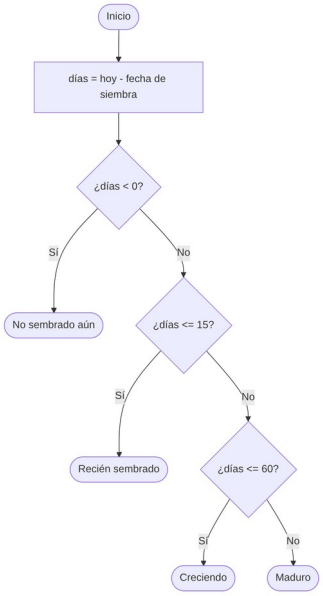
3. Cálculo de la etapa de crecimiento

Explicación. A partir de la fecha de siembra se calculan los días transcurridos y se clasifica el cultivo: **no sembrado** (días negativos), **recién sembrado** (0 a 15 días), **creciendo** (16 a 60) o **maduro** (más de 60). Se recalcula cada vez, así que se ajusta sola con el tiempo. El plan de agua usa la misma idea para asignar una **prioridad** de 1 a 10 (recién sembrado = 10, creciendo = 6, maduro = 3, no sembrado = 1).

Pseudocódigo

```
FUNCION calcular_etapa(fecha_siembra)
    dias = DIAS(HOY - fecha_siembra)
    SI dias < 0: DEVOLVER "no sembrado"
    SI dias <= 15: DEVOLVER "recién sembrado"
    SI dias <= 60: DEVOLVER "creciendo"
    DEVOLVER "maduro"
```

Diagrama de flujo



Complejidad. Tiempo O(1), espacio O(1).

Justificación. Evita que el usuario tenga que indicar la etapa a mano y mantiene los datos al día.

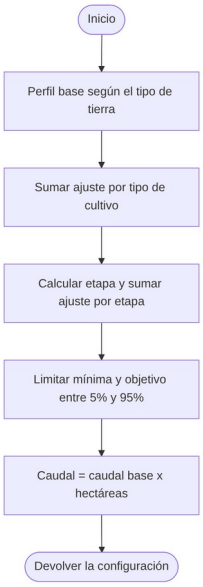
4. Configuración de riego (multicriterio)

Explicación. Para cada cultivo calcula la humedad mínima, la humedad objetivo y el caudal. Parte de un perfil según el tipo de tierra (arcillosa, arenosa, franca o limosa), le suma un ajuste por tipo de cultivo y otro por etapa (un cultivo recién sembrado necesita más humedad mínima), limita los valores entre 5% y 95% y escala el caudal según el tamaño de la parcela (en hectáreas).

Pseudocódigo

```
FUNCION calcular_configuracion_zona(cultivo)
    perfil = PERFIL_TIERRA(cultivo.tipo_tierra)
    ajuste_cultivo = AJUSTE_TIPO(cultivo.tipo)
    etapa = calcular_etapa(cultivo.fecha_siembra)
    minima = perfil.minima + ajuste_cultivo + AJUSTE_ETAPA(etapa)
    objetivo = perfil.objetivo + ajuste_cultivo
    minima = LIMITAR(minima, 5, 95)
    objetivo = LIMITAR(objetivo, minima + 5, 95)
    caudal = CAUDAL_BASE_POR_HECTAREA * cultivo.tamano_parcela
    DEVOLVER (minima, objetivo, caudal, etapa)
```

Diagrama de flujo



Complejidad. Tiempo O(1) por cultivo (tres búsquedas en diccionarios pequeños y operaciones fijas). Espacio O(1).

Justificación. Un modelo de ajustes sumados es transparente, fácil de calibrar por un agrónomo y no necesita datos históricos. Cada factor se puede cambiar de forma independiente.

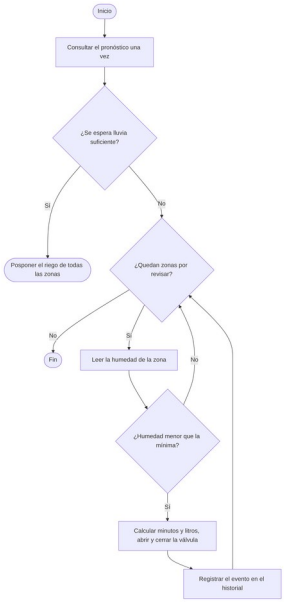
5. Decisión de riego (humedad y clima)

Explicación. Al revisar todos los cultivos, consulta el pronóstico una sola vez. Si se espera lluvia suficiente, pospone el riego de todas las zonas. Si no, recorre cada zona: lee la humedad simulada y, si está por debajo del mínimo, calcula los minutos de riego (con un factor de reducción si hay probabilidad de lluvia leve), abre la válvula, calcula los litros, actualiza la humedad, cierra la válvula y registra el evento en el historial de esa zona.

Pseudocódigo

```
FUNCION revisar_y_regar_todas(zonas, clima)
    pronostico = clima.obtener_pronostico()
    SI pronostico.prob_lluvia >= UMBRAL_PROB Y pronostico.mm >=
UMBRAL_MM
        POSPONER riego de todas las zonas ; SALIR
    PARA CADA zona EN zonas
        humedad = zona.sensor.leer()
        SI humedad < zona.config.minima
            factor = SI pronostico.prob_lluvia >= 0.3 ENTONCES 0.7
SINO 1.0
            minutos = LIMITAR(((objetivo - humedad) / 2) * factor, 1,
maximo)
            ABRIR valvula ; litros = minutos * caudal
            ACTUALIZAR humedad ; CERRAR valvula
            REGISTRAR evento en el historial de la zona
```

Diagrama de flujo



Complejidad. Tiempo O(n), con n cultivos (el clima se consulta una sola vez). Espacio O(n) por los eventos del historial.

Justificación. La decisión de cada cultivo es independiente de la de los demás, así que un recorrido lineal es lo más simple y eficiente. Con decenas o cientos de cultivos es más que suficiente.

6. Recomendación de agua (plan de agua)

Explicación. `recomendar` calcula cuántos litros necesita un cultivo y cada cuántos días regarlo. Usa tablas de referencia: milímetros de agua por riego según el cultivo (por ejemplo maíz 25, frijol 20, arroz 50), días entre riegos, un factor por tipo de tierra (arenosa 1.25, arcillosa 0.85, franca 1.0, limosa 0.9) y un ajuste de días por tierra. Como 1 mm de agua sobre 1 hectárea equivale a 10,000 litros, los litros son: hectáreas × mm × 10,000 × factor de tierra.

Pseudocódigo

```
FUNCION recomendar(cultivo)
    mm = MM_POR_RIEGO[tipo] o 20 si no existe
    litros = hectareas * mm * 10000 * FACTOR_TIERRA[tierra]
    dias = DIAS_ENTRE_RIEGOS[tipo] + AJUSTE_DIAS_TIERRA[tierra]
(mínimo 2)
    (etapa, prioridad) = calcular_etapa(fecha_siembra)
    DEVOLVER (litros, dias, etapa, prioridad)
```

Complejidad. Tiempo O(1) por cultivo, espacio O(1).

Justificación. Tablas simples y editables que cualquier persona puede entender y ajustar con datos agronómicos más exactos.

7. Cola de riegos pendientes

Explicación. Una **cola** es una estructura FIFO: el primero que entra es el primero que sale. Cada cultivo que el usuario agrega a pendientes se encola (si ya estaba, el programa avisa). Al marcar un riego como hecho, se desencola el primero y pasa al historial.

Pseudocódigo

```
FUNCION programar_riego(codigo)
    SI codigo YA ESTA en la cola: DEVOLVER falso
    ENCOLAR codigo ; GUARDAR ; DEVOLVER verdadero

FUNCION atender_siguiente()
    codigo = DESENCOLAR()
    SI codigo es NADA: DEVOLVER NADA
    AGREGAR AL INICIO del historial {codigo, fecha y hora}
```

Complejidad. Encolar y desencolar O(1); comprobar si ya está en la cola O(n). Espacio O(n).

Justificación. Los riegos se atienden en el orden en que se programaron, que es justo el comportamiento de una cola.

8. Historial con lista enlazada

Explicación. Cada riego hecho se agrega **al inicio** de una lista enlazada, así el más reciente aparece primero. Para saber cuándo fue el último riego de un cultivo se recorre la lista hasta el primer registro de ese código.

Complejidad. Agregar al inicio O(1); recorrer o buscar el último riego O(n). Espacio O(n).

Justificación. Insertar al inicio no requiere mover elementos, y siempre se consulta primero lo más reciente.

9. Reparto de agua – algoritmo voraz

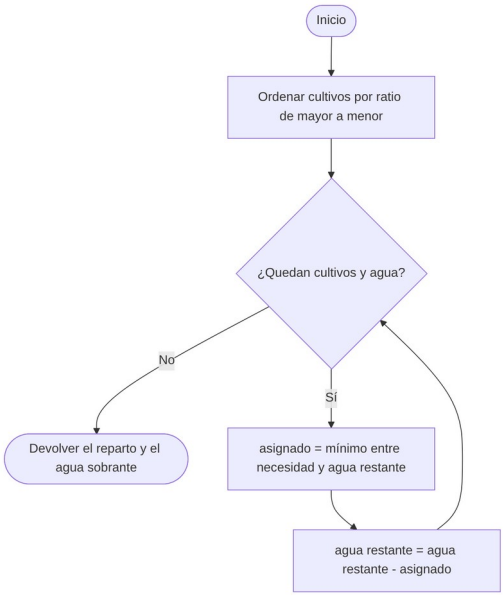
Explicación. Con una cantidad limitada de litros, se calcula para cada cultivo su **ratio** = valor / necesidad (valor = prioridad × hectáreas). Se ordenan de mayor a menor ratio con Merge Sort y se reparte agua en ese orden, dando a cada cultivo lo que necesita o lo que quede. El último puede recibir una parte (como en la mochila fraccionaria), donde la estrategia voraz garantiza el óptimo.

Pseudocódigo

```
FUNCION repartir_agua(items, agua_total)
    ordenados = MERGE_SORT(items, por ratio) de mayor a menor
    restante = agua_total
    PARA CADA item EN ordenados
        item.asignado = MINIMO(item.necesidad, restante)
```

```
        restante = restante - item.asignado
    DEVOLVER (ordenados, restante)
```

Diagrama de flujo



Ejemplo real (4000 L). Frijol 1500 de 1500 L; maíz 2000 de 2000 L; sandía 500 de 3000 L (17%); sobra 0 L.

Complejidad. Tiempo O(n log n) (por el ordenamiento), espacio O(n).

Justificación. Es una estrategia rápida, fácil de explicar y óptima para este tipo de problema.

10. Distancias y recorridos – grafo y Dijkstra

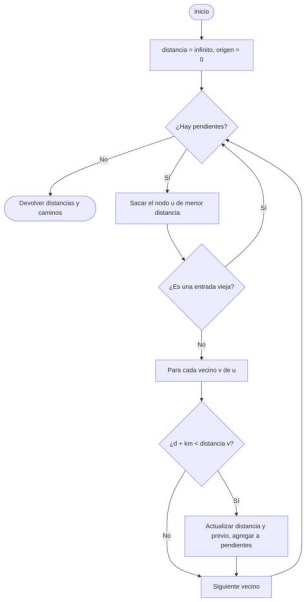
Explicación. Los lugares (la casa y cada parcela) son los **nodos** de un grafo y las distancias en kilómetros son las **aristas**. Se guarda como diccionario de vecinos. **Dijkstra** calcula la distancia más corta desde la casa a los demás lugares, usando una cola de prioridad, y guarda de qué lugar viene cada uno para reconstruir el camino.

Pseudocódigo

```
FUNCION dijkstra(grafo, origen)
    distancia[todos] = INFINITO ; distancia[origen] = 0 ;
previo[todos] = NADA
    pendientes = [(0, origen)]
    MIENTRAS pendientes no esté vacío
        (d, u) = SACAR el de menor distancia
        SI d > distancia[u]: CONTINUAR
        PARA CADA (v, km) VECINO DE u
            SI d + km < distancia[v]
                distancia[v] = d + km ; previo[v] = u
```

```
                AGREGAR (distancia[v], v) A pendientes
    DEVOLVER distancia, previo
```

Diagrama de flujo



Ejemplo real. Casa–A 10 km, Casa–B 20 km, A–B 5 km: a B hay 15 km por Casa → A → B.

Complejidad. Tiempo $O((V + E) \log V)$, espacio $O(V + E)$.

Justificación. Las distancias son positivas, que es la condición de Dijkstra. Devuelve el camino más corto en kilómetros entre parcelas lejanas.

11. Mejor orden para visitar los pendientes

Explicación. `orden_de_visita` usa una estrategia voraz del **vecino más cercano**: desde la casa, va siempre a la parcela pendiente más cercana (calculada con Dijkstra) y repite desde ahí. Devuelve el orden, los kilómetros totales y las parcelas sin camino registrado. No siempre da el recorrido óptimo, pero es rápido y fácil de explicar.

Pseudocódigo

```
FUNCION orden_de_visita(pendientes)
    actual = CASA ; orden = [] ; total = 0
    MIENTRAS queden pendientes alcanzables
        distancias = dijkstra(grafo, actual)
        siguiente = el pendiente alcanzable de menor distancia
        total = total + distancias[siguiente]
        AGREGAR siguiente A orden ; QUITAR siguiente de pendientes
        actual = siguiente
    DEVOLVER (orden, total, pendientes sin camino)
```

Complejidad. Tiempo $O(k \cdot (V + E) \log V)$, con k pendientes. Espacio $O(V + E)$.

12. Guardado y manejo de errores

El estado del sensor y los riegos de las zonas se guardan en `datos_riego.json`; los pendientes, el historial y las distancias, en `datos_optimizacion.json`. Si un archivo está dañado o no existe, el programa arranca vacío en lugar de cerrarse. La pantalla valida que haya un cultivo seleccionado, que los litros sean un número mayor que cero y que las distancias sean positivas y entre lugares distintos, y muestra mensajes en rojo cuando algo falla.